

Article

Real-Time Passenger Flow Analysis in Tram Stations Using YOLO-Based Computer Vision and Edge AI on Jetson Nano

Sonia Diaz-Santos ^{*,†} , Pino Caballero-Gil [†]  and Cándido Caballero-Gil [†] 

Department of Computer Engineering and Systems, University of La Laguna, Camino San Francisco de Paula, 38200 San Cristóbal de La Laguna, Spain; pcaballe@ull.edu.es (P.C.-G.); ccabgil@ull.edu.es (C.C.-G.)

* Correspondence: sdiazsan@ull.edu.es

† These authors contributed equally to this work.

Abstract

Efficient real-time computer vision-based passenger flow analysis is increasingly important for the management of intelligent transportation systems and smart cities. This paper presents the design and implementation of a system for real-time object detection, tracking, and people counting in tram stations. The proposed approach integrates YOLO-based detection with a lightweight tracking module and is deployed on an NVIDIA Jetson Nano device, enabling operation under resource constraints and demonstrating the potential of edge AI. Multiple YOLO versions, from v3 to v11, were evaluated on data collected in collaboration with Metropolitano de Tenerife. Experimental results show that YOLOv5s achieves the best balance between detection accuracy and inference speed, reaching 96.85% accuracy in counting tasks. The system demonstrates the feasibility of applying edge AI to monitor passenger flow in real time, contributing to intelligent transportation and smart city initiatives.

Keywords: computer vision; YOLO; real-time object detection; people counting; edge AI; intelligent transportation systems; Jetson Nano



Academic Editor: Paolo Bellavista

Received: 6 October 2025

Revised: 29 October 2025

Accepted: 30 October 2025

Published: 3 November 2025

Citation: Diaz-Santos, S.; Caballero-Gil, P.; Caballero-Gil, C. Real-Time Passenger Flow Analysis in Tram Stations Using YOLO-Based Computer Vision and Edge AI on Jetson Nano. *Computers* **2025**, *14*, 476. <https://doi.org/10.3390/computers14110476>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Public transport is a fundamental part of the infrastructure of any modern city. In particular, tram systems play a crucial role in urban mobility, offering an efficient, sustainable, and accessible transportation option. However, one of the most significant challenges facing transport authorities and tram operators is the effective management of passenger flow. In many cities, high passenger density during peak hours can lead to congestion, delays, and a negative user experience.

Tram stations, especially in densely populated urban areas, experience significant fluctuations in ridership throughout the day. These peak demands can cause operational problems such as overloaded cars, extended waiting times, and difficulties in boarding and alighting. These problems affect the efficiency of the transportation system and negatively impact user satisfaction and overall perception of public transportation.

Real-time passenger flow analysis using artificial intelligence and computer vision offers an innovative and effective solution to address these challenges. By implementing advanced systems that can continuously monitor and analyze passenger flow, it is possible to improve the management and operation of tram services in a number of ways. With this accurate real-time data, operators can adjust tram frequency and vehicle allocation more efficiently, responding dynamically to fluctuations in demand and optimizing resources.

A more efficient and predictable transportation system significantly improves user satisfaction, reduces waiting times, and provides a more comfortable and safer ride. However, the ability to monitor and analyze passenger flow also improves safety, enabling a faster and more effective response in emergency situations or evacuations. Renfe has deployed a real-world example of such a passenger monitoring system at one of its train stations, as reported in [1].

Managing passenger flow in tram stations and other public transport systems has been a persistent challenge throughout history. With the growth of cities and the acceleration of urbanization, public transport has become an essential backbone of urban mobility.

In the early stages of public transport, the management of passenger flow relied mainly on manual methods and direct observations. The tram and bus operators counted passengers by hand, recording the data on sheets of paper. As cities grew and the demand for public transport increased, these manual methods began to show their limitations. The capacity to handle large volumes of passenger data became inadequate, and the precision of manual observations was low, making efficient service planning and management difficult.

In the 1970s and 1980s, the first automatic passenger counting (APC) systems were introduced. These systems used basic sensors, such as infrared barriers and pressure-sensitive mats, installed at the entrances and exits of transport vehicles. These sensors could count the number of passengers boarding and alighting the tram, providing more accurate and real-time data than manual methods. Despite the improvements, these early APC systems had their limitations. Infrared sensor technology, for example, could be inaccurate in high-passenger density conditions and could not distinguish between different types of objects. In addition, these systems lacked the ability to analyze complex patterns of passenger movement and behavior. As technology advanced in the 1990s and 2000s, APC systems became more sophisticated. More advanced sensors were introduced, such as video cameras and radar systems, which could provide more accurate and detailed data. In addition, the proliferation of wireless communication technologies enabled real-time data transmission to centralized control centers. These advances enabled better integration of passenger data with other transportation management systems, such as scheduling and fleet management. Transport operators could now monitor passenger flow in real time and make informed decisions to adjust service according to demand.

The digital revolution and the rise of Big Data since 2010 have radically transformed passenger flow management. With the ability to process large volumes of data in real time, transport operators began to use Big Data analytics to identify patterns and trends in public transport usage.

Computer vision has been one of the most significant advances in passenger flow management. Using video cameras and advanced image processing algorithms, these systems can detect and track passenger movement in real time. This has enabled greater accuracy in identifying congestion and optimizing passenger flows. These systems use convolutional neural networks (CNN) for image recognition and object detection. In addition, they can differentiate between passengers and other objects, such as luggage or bicycles, and provide detailed data on passenger behavior, such as waiting times, preferred routes, and movement patterns.

In recent years, numerous projects and implementations of advanced passenger flow management systems have been carried out around the world. For example, in New York, the MetroFlow project used computer vision and Big Data analytics to improve subway management. This project demonstrated how AI algorithms could improve passenger screening accuracy and provide real-time data for operational decision making. In London, Transport for London (TfL) has deployed advanced monitoring systems in underground and bus stations, combining sensors, cameras, and AI-based tracking algorithms. These

systems enable real-time analysis of bidirectional passenger flows in station passageways, improving service efficiency and enhancing user experience [2].

Common techniques and algorithms in video analysis and object detection represent crucial areas in computer vision, with applications ranging from security and surveillance to autonomous vehicles and augmented reality. These techniques focus on detecting and tracking objects of interest in real-time or recorded video sequences, enabling a deeper interpretation and understanding of the visual scene. In the context of passenger flow analysis, the main applied problem can be formulated as a combination of person detection and real-time counting, where the goal is to identify each individual in a video frame and accurately estimate the number of people entering or leaving a predefined area. This task differs from flow density estimation or anomaly detection, as the objective here is to obtain precise counts and movement trajectories to evaluate crowd dynamics.

Among the main algorithms developed for this purpose, convolutional neural networks (CNNs) form the foundation for most modern object detectors. Architectures such as You Only Look Once (YOLO) and the Single Shot MultiBox Detector (SSD) extend CNN-based designs to achieve real-time inference on video streams. Although transformer-based detectors (e.g., DETR and its derivatives) have recently emerged and provide strong performance in complex scenes, their computational cost makes them less suitable for deployment on low-power edge devices like the Jetson Nano used in this work. For this reason, the proposed system focuses on lightweight CNN-based approaches that balance accuracy and inference speed in constrained environments. Nevertheless, future research may explore transformer-based approaches once computational constraints are mitigated.

The relevance of these methods extends beyond public transportation to environments such as shopping malls, security facilities, or public events, where real-time analysis of human flow provides valuable insights for resource allocation, safety enhancement, and service optimization.

This research differs from previous studies by addressing the general problem of real-time passenger flow analysis through the integration of artificial intelligence and computer vision techniques optimized for edge computing environments. The proposed system focuses on the accurate detection, tracking, and counting of individuals in real-time video streams, aiming to improve operational efficiency in public transportation systems. The scientific contribution lies in demonstrating how lightweight deep learning models, such as YOLO for detection and DeepSORT for tracking, can be effectively implemented on low-power embedded devices like the NVIDIA Jetson Nano while maintaining competitive accuracy and inference speed.

The Tenerife tram network has been selected as a representative test case to validate the system in a real-world setting. This environment provides a relevant context for assessing system robustness under varying lighting conditions, passenger densities, and background complexity. However, the methodological framework developed in this work is broadly applicable to similar transportation infrastructures and can be readily adapted to other urban environments.

The results obtained are expected to improve passenger flow management in the selected case study and to advance the broader field of computer vision-based intelligent transportation systems by demonstrating a scalable, energy-efficient approach for real-time people detection and counting at the edge.

The main contributions of this work are the development of a YOLO-based detection pipeline optimized for deployment on Jetson Nano, the design of a lightweight tracking module for real-time passenger monitoring, an empirical benchmark of YOLO variants from v3 to v11 on real-world tram station data, and the validation of the system in collaboration with Metropolitano de Tenerife.

This paper is structured as follows. Section 2 provides context through a review of previous research and a discussion of different technical aspects. Subsequently, Section 3 outlines relevant alternative use cases where this project could also be applied. Section 4 describes the requirements and technologies involved in this project. Section 5 presents the system architecture scheme. Section 6 delves into practical implementation details and explains in depth the functionality of the program. Section 7 shows and explains the results obtained. Finally, Section 8 summarizes the main conclusions and their implications.

2. Related Work

Passenger flow analysis in public transport is a field of research and development that has gained great relevance in recent decades. The ability to monitor and analyze passenger flow in real time provides essential information to optimize the operation and planning of public transport, improving efficiency and user experience. In recent years, there has been significant growth in the field of human detection and counting, driven by advances in computer vision algorithms and machine learning techniques. The research by Viola and Jones in 2001 [3] introduced an effective method for detecting objects in images, laying the groundwork for further advancements in this field.

Convolutional neural networks have completely transformed the field of computer vision in the last decade. Notably, models like Faster R-CNN [4], YOLO [5], and SSD [6] have played a crucial role in this revolution. These models have demonstrated exceptional accuracy and speed in detecting objects, including individuals. As a result, they have become essential tools in human detection systems.

In the field of people counting, methods like density regression and heat maps have gained attention for accurately counting people in crowded images. CSRNet [7] and SANet [8] have shown their effectiveness in surpassing traditional feature-based methods. These methods are particularly efficient when analyzing large groups of people in dense environments. By integrating object tracking techniques, systems like DeepSORT [9] can accurately track individuals in densely populated environments, allowing for a thorough analysis of human behavior.

A new development in the field of crowd counting in large areas involves the use of fisheye cameras installed on ceilings. These cameras have a wide field of view and solve the problem of objects blocking the view, which is common with regular cameras. This makes fisheye cameras especially useful for crowded environments like train stations and malls. According to a recent study [10], when these cameras are combined with advanced computer vision algorithms, they can accurately count people even in crowded places.

Recent research [11] has investigated the use of image processing and artificial neural networks to precisely calculate the number of people in public transportation systems. This approach can adapt to different circumstances and improve capacity management. As a result, it leads to a significant improvement in user satisfaction. Similar concerns are addressed in vehicular networks, where packet scheduling strategies improve video delivery performance under dynamic conditions [12].

A research paper [13] introduces a technique that utilizes compute unified device architecture (CUDA) to take advantage of the parallel computing capabilities of graphics processing units (GPUs) for instant people counting. This strategy notably improves the speed and effectiveness of detection and counting. It is particularly useful in crowded settings such as tram stations in Tenerife, where real-time data processing is crucial. In the context of urban rail transit, service-driven routing algorithms have also been proposed to enhance communication efficiency and passenger information systems [14].

Significant advancements in real-time object detection and tracking have been achieved with YOLO on Jetson Nano, particularly in counting individuals and bicycles. This

study [15] presents a robust approach that proves the feasibility of using state-of-the-art computer vision systems on energy-efficient devices like Jetson Nano. This development shows that it is possible to achieve precision and efficiency even with energy-constrained devices. Such approaches are particularly beneficial for real-time surveillance and monitoring in transport systems. Reinforcement-learning-based orchestration of edge offloading has also been studied as a promising approach to optimize resource allocation in computing continuum environments [16].

A different study [17] presents a motion detection algorithm that uses a counter-propagation artificial neural network (CPN) specifically designed for static camera surveillance. This approach aims to improve the precision and effectiveness of motion detection. By incorporating CPN, the algorithm adeptly addresses the diverse obstacles encountered in motion detection.

Moreover, an effective approach for counting people indoors has been demonstrated using an adaptive counting system that incorporates Edge AI computing [18]. This method utilizes edge computing to process data locally, reducing latency and bandwidth usage. Such improvements are crucial for real-time applications in places like tram stations, where delays can negatively affect service quality.

Recent research on edge-centric transportation analytics further reinforces the potential of lightweight CNN-based models for onboard and roadside deployments [19].

Another important aspect to consider is the interrelationship between communication and computation in people counting systems, especially when using intelligence partitioning [20]. This research highlights the importance of effectively managing resources and system performance, which are crucial for applications that need real-time monitoring and high performance, such as tram station surveillance. Moreover, the importance of systematic testing and validation of computer vision systems has been highlighted as essential to ensure robust deployment in real-world applications [21].

These advances and studies emphasize the importance of combining computer vision and deep learning techniques with practical implementation strategies for accurate people counting and detection systems. Implementing these systems in Tenerife's tram stations can improve passenger monitoring and optimize resource allocation and strategic planning in the public transportation network.

Passenger flow analysis in public transportation has evolved from traditional counting methods to intelligent computer vision systems capable of detecting, tracking, and quantifying individuals in real time. In this work, the main objective is to evaluate the performance of lightweight detection models in accurately identifying and counting passengers under real operating conditions. To assess model performance, the work employs the metrics Precision, Recall, *F1*-score, and Accuracy, as they provide complementary information about detection reliability.

Although mean Average Precision (mAP) is commonly used in general object detection tasks to measure localization accuracy, the purpose of this research is not fine-grained localization but accurate counting of detected individuals across video frames. For this reason, the selected metrics emphasize the correctness of detections and their correspondence to true passenger instances rather than bounding-box precision. The mathematical definitions of these metrics, as applied in this study, are presented in Section 7.

This approach aligns the evaluation process with the research goal of achieving real-time, resource-efficient passenger detection and counting, while providing a methodological basis that can be extended to other transportation monitoring scenarios.

3. Use Cases

In this section we will explain the most relevant use cases in which this system can be very useful, providing more information and security to the establishment that uses it. Initially, we will delve into the primary use case for the system's development, which involves monitoring the crowd levels at tram stations. This enables the establishment to maintain a comprehensive record of the crowd at each station, along with an overall count of individuals across all stations. This data is further segmented by individual station counters and a global counter, as illustrated in Figure 1. This aspect plays a crucial role, as it not only enables improved services for tram users but also fortifies the security of the infrastructure. Furthermore, the system allows for tracking the number of individuals on each tram by monitoring entries and exits, thereby providing insights into crowd distribution at different stations and on various tram routes.



Figure 1. Use case: Tram stations in Tenerife.

In this way, overload situations can be avoided, improving the user experience and optimizing the distribution of resources. In the event that a large crowd is identified at a particular station, it is possible to enhance the frequency of trams serving that station. This adjustment can effectively decrease waiting times and prevent overcrowding. In addition, in terms of security, having accurate real-time data allows for a faster and more efficient response in case of emergencies, such as evacuations or security incidents.

An additional practical application involves implementing this system within shopping malls. By utilizing the people counting system, managers can gain valuable insights into customer traffic patterns across various sections of the mall, enabling them to enhance the allocation of security and cleaning personnel, and tailor marketing and sales tactics according to customer preferences. Furthermore, during busy periods or special occasions like holidays, real-time monitoring of crowd sizes in different zones can be conducted to uphold visitor safety and satisfaction.

Security facilities, including police stations and detention centers, can utilize the system to effectively oversee the movement of individuals and identify any abnormal trends that may suggest potential security issues. One such instance could be the identification of an unanticipated surge in the population within a particular region, which could serve as an indication of an ongoing incident demanding immediate intervention.

In parks, the system can be utilized to monitor the arrival of visitors, thereby enabling authorities to efficiently manage resources and ensure the safety of park users. By obtaining precise data on the number of people in various sections of the park during special events

or days with high footfall, overloading can be prevented and the presence of sufficient staff can be ensured.

Accurate people counting is crucial in healthcare settings for monitoring patient and visitor traffic. This technology helps hospital and clinic administrators allocate staff and resources effectively, improving operational efficiency and reducing patient wait times. Additionally, real-time monitoring during emergencies or outbreaks allows for quick tracking of individuals in critical areas, facilitating coordinated responses.

This system offers significant advantages in managing resources, improving security, and increasing user satisfaction in important settings like airports, stadiums, and government buildings. By accurately and instantly counting people in real-time, it enables authorities and administrators to make informed decisions quickly, leading to improved operational efficiency and enhanced overall facility security.

4. Requirements and Technologies

This section will discuss the hardware and technologies used in this project. To begin with, the fundamental piece of work is an NVIDIA Jetson Nano development kit, 4 GB model, which is a high-performance computing platform designed for artificial intelligence and deep learning applications such as image classification, object detection, segmentation and natural language processing. This small but powerful computer offers the ability to run complex neural networks and perform real-time data processing tasks, all in a compact and energy-efficient form factor.

The Jetson Nano features a 128 CUDA-core Nvidia Maxwell processor, enabling efficient parallel processing calculations. It also includes a quad-core ARM Cortex-A57 central processing unit (CPU). It has 4 GB of LPDDR4 memory, which is suitable for handling AI and deep learning tasks. It has a microSD card slot for OS and data storage. It also has expansion interfaces for adding additional storage if needed. It offers several connectivity options: four USB 3.0 ports, a USB 2.0 Micro-B port, a Gigabit Ethernet port, an HDMI port, a barrel power connector (5 V 4 A) and a 40-pin GPIO connector for integration with other devices and sensors. Supports 4K video at 30 fps and 1080p at 60 fps, making it ideal for computer vision and real-time video processing applications. Its operating system is Nvidia JetPack SDK, which includes an Ubuntu-based operating system and a collection of tools and libraries for developing AI and deep learning applications. It is ideal for a wide variety of applications, including autonomous robots, drones, intelligent video surveillance systems, real-time video analytics, internet of things (IoT) devices, and any project requiring advanced computing capabilities. The Jetson Nano development kit has the specifications [22] shown in Table 1.

Then the necessary technologies used to generate this project will be discussed. Python has been used as a programming language because it is a high-level programming language. It is widely used in the development of artificial intelligence applications, data processing, and many other areas due to its extensive standard library and the support of a large community.

The libraries used are: Open Source Computer Vision Library (OpenCV) for image and video processing, as it is an open source computer vision library that provides more than 2500 optimised algorithms. It is ideal for tasks such as facial recognition, object detection and general image processing. Pandas for data manipulation and analysis because it is a powerful and flexible library that provides high-performance data structures, such as dataframes, which are essential for the analysis and manipulation of tabular and heterogeneous data. It is very popular in data analysis, statistics and machine learning. Numerical Python (Numpy) for numerical operations as it is a fundamental library for scientific computing in Python. It provides support for large multidimensional arrays and

matrices, along with a collection of high-level mathematical functions to operate on this data efficiently. Ultralytics YOLO for object detection as it is an advanced implementation of the YOLO algorithm, which is one of the most popular methods for real-time object detection. This library enables accurate and fast detections and is suitable for applications in security, automation and video analytics.

Table 1. Jetson Nano 4 GB Development Kit Specifications.

Component	Description
GPU	NVIDIA Maxwell™ architecture 128 NVIDIA CUDA cores
CPU	Quad-core ARM® Cortex®-A57 MPCore processor
Memory	4 GB 64-bit LPDDR4
Storage	16 GB eMMC 5.1 Flash storage
Video Encoding	4K at 30 fps (H.264/H.265)
Video Decoding	4K at 60 fps (H.264/H.265)
Camera Input	12 lanes (3 × 4 or 4 × 2) MIPI CSI-2 DPHY 1.1 (18 Gbps)
Connectivity	Gigabit Ethernet
Display Output	HDMI 2.0 or DP 1.2/eDP 1.4/DSI (1 × 2) 2 simultaneous
UPHY	1 1/2/4 PCIE, 1 USB 3.0, 3 USB 2.0
I/O	1 SDIO/2 SPI/4 I2C/2 I2S/GPIO
Dimensions	69.6 mm × 45 mm
Mechanical	260-pin connector

YOLO-based architectures were evaluated in this study as the primary detection framework due to their balance between inference speed and accuracy in real-time object detection tasks. Several YOLO variants, from v3 to v11, were tested under the same experimental conditions to determine the most suitable model for deployment on the Jetson Nano device. The evaluation was carried out using a dataset composed of video sequences recorded at Tenerife tram stations and additional open-source footage representing similar passenger flow conditions in public transportation environments. The dataset includes diverse lighting and crowd density scenarios to ensure realistic testing conditions.

The selection of YOLOv5s as the detection model was guided by experimental results that demonstrated its superior balance between detection accuracy, inference speed, and computational efficiency on the Jetson Nano platform. Among the tested versions, YOLOv5s consistently provided the best trade-off between accuracy and processing speed while maintaining stable performance under resource constraints. Its lightweight architecture enables the detection and localization of multiple individuals per frame with minimal latency, making it particularly suitable for embedded real-time applications.

The comparative evaluation methodology, including the mathematical definitions of the metrics, is presented in Section 7. These metrics collectively assess both the correctness and efficiency of the detection process, ensuring a fair and transparent model comparison.

5. System Architecture

To address the advancement and evaluation of the system for the detection and counting of individuals in the Tenerife tram stations, a methodology based on the use of a Jetson Nano was used together equipped with a camera module. These tools are used to capture images in real time and perform passenger detection. This approach employs object tracking techniques and people detection methods. To ensure feasibility, a dataset

collected from tram stations was used during the implementation of these techniques. This allows customizing and improving the system to adapt it to the real conditions of the environment.

The system architecture of this project is detailed below for a better understanding of the implementation process. In addition, the complete structure of the system, as well as the communication between all the elements of the system, can be seen in Figure 2. The system is designed with a modular architecture that allows for easy integration and maintenance. The main components of the system include:

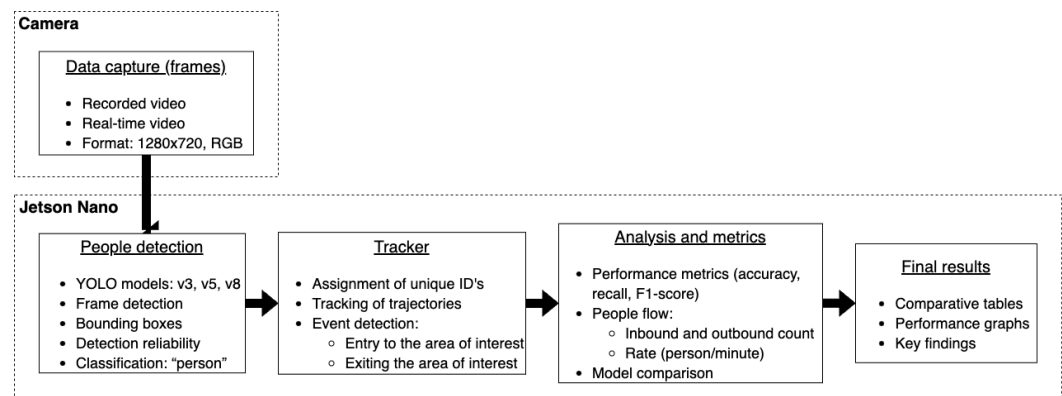


Figure 2. System architecture.

1. **Data Capture:** First, data capture takes place where initially pre-recorded videos of people flow will be used to train and validate the system. These videos simulate real tram station conditions and provide a solid basis for model development. In the final phase of the project, a camera connected to a Jetson Nano is integrated to capture real-time images at the tram station. The OpenCV library is used, which provides robust tools for video capture and processing. OpenCV takes care of capturing frames of the input video on a continuous basis, allowing real-time analysis. This component is crucial as it provides the raw data on which object detections and tracking will be performed.
2. **People Detection:** Once the images have been obtained from the pre-recorded videos in the initial phase or captured in real time in the final phase, the second step is the detection of passengers using the YOLO object detection model. YOLO is known for its high speed and accuracy in object detection, which makes it ideal for real-time applications. This model is trained and adjusted to identify passengers in the tram station images, detecting and delimiting each person with detection frames. The second step is the count of people. This process will involve the identification and counting of each detected passenger in the bounding boxes. The count will be updated in real time to provide accurate information on the number of passengers present at any given moment. In order to count people entering and leaving a certain area, specific areas are defined within the frame. These areas of interest are essential for people flow analysis. The system monitors these areas to detect when a person crosses the defined boundaries, allowing for an accurate count of entries and exits.
3. **Tracker:** Subsequently, in order to have the knowledge of how many people enter and leave the station, a tracking system is implemented. For this purpose, this system tracks each passenger detected over time, using tracking algorithms based on euclidean distance. The tracker allows a continuous and up-to-date count of incoming and outgoing passengers to be maintained, providing valuable data on the flow of people in the station.

4. Analysis and metrics: Finally, statistics are generated indicating the number of people who have entered and left the defined areas of interest. These statistics can be printed on screen and saved to a file for later analysis. The data obtained from passenger counting and tracking is analyzed and visualized through interactive dashboards. These dashboards display real-time information on the number of passengers arriving and departing, helping transport operators better manage capacity and resources.

Each of these components is integrated in a modular way, allowing the system to be flexible and scalable. This modular architecture also facilitates the maintenance and upgrade of the system, as each component can be modified or upgraded independently without affecting the others.

6. Implementation

The program is divided into two main parts: the main script and the tracker module. All related scripts, configurations, and evaluation data are available in the GitHub repository [23], where the specific code used in this study can be accessed. The complete results and values can also be found there. In this section we will explain the functionality of each part of the code for a better understanding of how it works.

6.1. Main Script

The main script is the central component of the program and takes care of most of the program's functionalities, including video capture, object detection, definition of areas of interest and real-time display of the results. The libraries are imported and, specifically, the YOLOv5s model, which is used for object detection in the video frames.

To start the program, we define the area of interest (AOI) used to register the entries and exits of people. One of the areas of interest is a blue rectangle whose main purpose is to serve as a delimiter to determine whether people are entering or leaving the tram; to define it, an array of coordinates is generated. Subsequently, a red line is established that corresponds to the line of the rectangle that is closest to the tram. These delimiters have several joint functionalities; to begin with, the rectangle is used to determine when a person has crossed the first line and is within the rectangle area when entering the tram. Subsequently, if the person is in the area of interest and, in addition, crosses the red line, the person is considered to be entering the tram. If a person is in the area of interest and exits, he/she is not taken into account since he/she cannot cross the red line without having been in the rectangle area.

Once we have these two delimiters, we start analyzing the frames of the pre-recorded video or sent through the camera. The Common Objects in Context (COCO) class detection file is loaded in order to detect people. The tracker object is initialized to be able to follow the detected people, which is updated as new people are detected.

Now, the delimiters are drawn in the frame being analyzed, both the area of interest and the red line. Then, only the class that detects people is filtered with the COCO file. The green colored bounding boxes that detect people are created and drawn together with their person ID and a central point on the bottom line, which is used to locate the person in the frame.

Subsequently, it is determined whether the person is within the blue rectangle of the area of interest and, if the person crosses the red line. Thus, if the person has crossed both delimiters, an entry to the tram is recorded. Finally, the counter displayed in the frame of people entering or exiting, as appropriate, is incremented.

6.2. Tracker

The tracking module, called tracker, is a fundamental part of the program that is responsible for tracking the people detected throughout the video frames. This module is invoked from the main script and provides several key functionalities.

First, an instance of the Tracker class is created, which is responsible for managing the tracking of objects. At each frame, new detections of people are passed to the tracker to update the status of the tracked objects. The tracker assigns a unique identifier to each person and maintains its state over time. The tracker uses the areas of interest defined in the main script to determine when a person enters or leaves a specific area. This is done by comparing the coordinates of the tracked objects with the areas of interest. Although most of the visual processing is done in the main script, the tracker provides the information needed to draw bounding boxes and identifiers about each person.

An object is considered to have entered the monitored area when its centroid (x_c, y_c) crosses the reference line $y = y_{\text{line}}$ in the downward direction within the defined region of interest (ROI). Conversely, a crossing in the upward direction is counted as an exit. Formally:

$$\text{if } y_{\text{prev}} < y_{\text{line}} \wedge y_{\text{curr}} \geq y_{\text{line}} \Rightarrow \text{entry}$$

$$\text{if } y_{\text{prev}} > y_{\text{line}} \wedge y_{\text{curr}} \leq y_{\text{line}} \Rightarrow \text{exit}$$

This simple geometric rule is implemented at tracking time to ensure consistent counting even under partial occlusion or minor camera motion.

In order to calculate the Euclidean distance between the centers of the detected objects, the math library is imported. The Tracker class is defined, which is used to manage the tracking of the detected objects. Then, the positions of the detected objects and a counter that is incremented each time a new object is detected are stored, providing a unique identifier for each person. An update method is created which is responsible for updating the positions of the detected objects and assigning unique identifiers to them. Now, the center point of each detected object is calculated and checked if the object has been detected before by comparing the Euclidean distance between the center of the current object and the centers of the stored objects. If the object has not been detected before, the new identifier will be able to identify it. The identifiers of the objects that are no longer present in the new list of detections are removed. Finally, the list of bounding rectangles of the objects together with their unique identifiers is returned.

The use of Euclidean distance [24] for tracking people is based on its computational simplicity and its ability to measure proximity between objects in a two-dimensional space. Euclidean distance, when tracking multiple objects, allows one to quickly associate detections in consecutive frames of the video, minimizing confusion between nearby individuals. This is especially useful when the motion of detected objects is smooth and predictable, which is the case in most surveillance or people counting environments.

There are other methods such as the Kalman Filter [25] or Siamese Neural Networks [26], but Euclidean distance offers advantages in terms of speed and efficiency, as it does not require future motion estimates or prior training of complex models. Although the Kalman Filter is good at predicting trajectories when there is occlusion or noise in the detections, its implementation can be unnecessarily complex in well-lighted environments with good detection. Deep learning-based methods can offer higher accuracy in highly dynamic scenarios, but require large volumes of labeled data and high computational power.

For this reason, Euclidean distance has been chosen as a balanced solution, allowing a fast and accurate assignment of identities within a tracking system while maintaining a

low computational cost and ensuring real-time performance, which is key for applications such as video people detection and counting.

6.3. Interaction Between Main Code and Tracker

The interaction between the main script and the tracker is essential for the correct functioning of the program. The main script captures video, detects people and defines areas of interest. Then, it passes the detections to the tracker, which is in charge of keeping continuous tracking of the detected people. This collaboration allows the program to detect people in each frame and also to track their movements over time and determine their interactions with the areas of interest. This is especially useful for security applications, behavioral analysis and people flow monitoring.

6.4. Comparison with Other Methods

The innovative strategy proposed for monitoring the number of individuals at tram stations stands out due to its incorporation of cutting-edge algorithms, particularly CNNs, which contribute to achieving remarkable precision and reliability in people counting [27].

In contrast to alternative techniques like motion sensors, which are vulnerable to external disturbances (e.g., temperature, weather, and occlusions) and typically have accuracy rates of around 85–90% under ideal conditions [28], or Radio Frequency Identification (RFID)-based systems that may encounter constraints related to coverage, scalability, and accuracy (typically having an accuracy of 80–95% depending on infrastructure [29]), the suggested approach demonstrates exceptional adaptability and efficiency across various operational settings and urban landscapes.

In terms of performance, YOLOv5s is expected to offer high detection accuracy and fast inference speeds, making it suitable for real-time applications in dynamic environments like tram stations. This model has been widely used in various object detection tasks due to its efficiency and reliability. Compared to motion sensors, which can be vulnerable to disturbances in crowded settings, and RFID systems, which face scalability challenges and often require costly infrastructure, YOLOv5s provides a more flexible solution.

The YOLOv5s approach is more flexible and efficient than alternatives, as it relies on computer vision through video surveillance, avoiding the need for costly infrastructure. Additionally, edge computing reduces network load, improving system responsiveness and making it a more efficient solution for high-capacity public transport systems.

The proposed approach prioritizes operational efficiency by using edge computing for on-site data processing at tram stations. This reduces network burden and improves system efficiency compared to centralized approaches. It also improves response time and passenger capacity in high-traffic public transportation. The real-time processing capabilities of YOLOv5s allow for immediate adjustments to service, such as increasing tram frequency during peak times, which traditional methods like motion sensors or RFID cannot achieve as quickly.

This method is cost-effective and low-maintenance compared to alternatives. Unlike RFID systems, which require expensive infrastructure (including tags, readers, and complex networks) and frequent upkeep (such as battery replacements and recalibration), or motion sensors that are prone to malfunctions and need regular calibration, the proposed system mainly relies on video surveillance cameras and edge computing devices. This significantly reduces both initial setup costs and long-term operational expenses.

The system's scalability and coverage make it highly effective in urban settings. With each tram station operating independently, it can easily expand to multiple stations without requiring significant infrastructure upgrades. This ensures comprehensive coverage and

the ability to manage high volumes of people in densely populated areas, making it an attractive solution for crowd management and optimizing urban transportation systems.

7. Results

This section presents the results obtained from the implementation and execution of the program. Initially, different use cases were evaluated with pre-recorded videos in other scenarios such as parks or subway station interiors. However, with the collaboration with the Metropolitano de Tenerife, it was possible to obtain real videos to test the operation of the program in the Jetson Nano.

Therefore, we first use a video of tram stations in real conditions. The results of the program execution can be seen in Figure 3. In it, you can see all the features that have been mentioned throughout the article; you can see how people are detected in a green rectangle with the identifier of the detected person at the top of the rectangle. You can also see the area of interest drawn with a blue rectangle, with the red line closest to the tram as the limit to count the entries and exits. In addition, the number of people entering and exiting the tram is displayed.

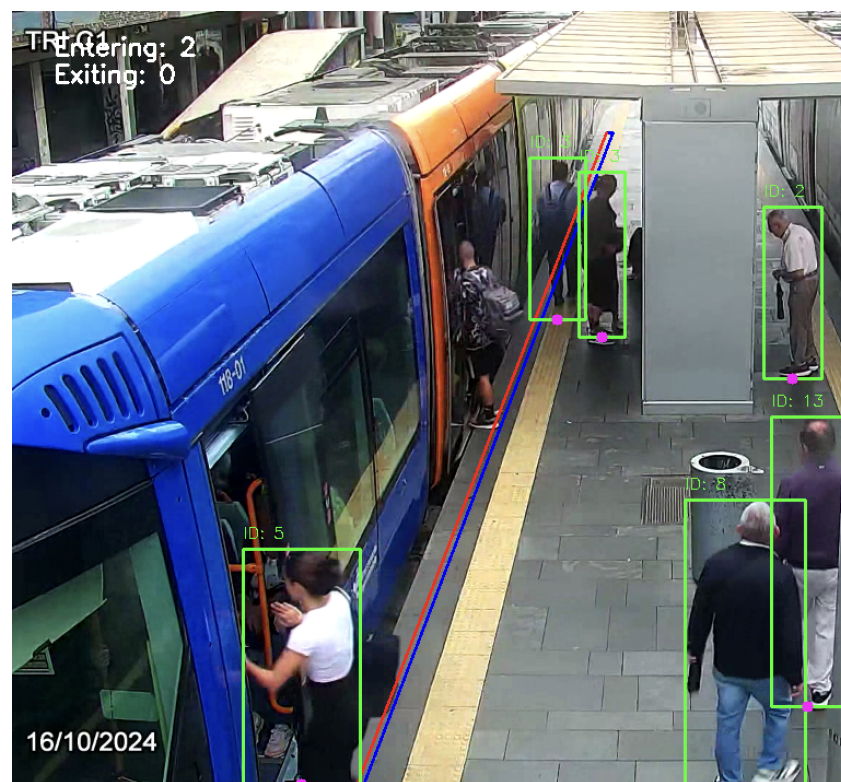


Figure 3. Results obtained at tram stations.

The placement of the camera at tram stations, as well as the definition of the boundaries of the area of interest and the red line, plays a crucial role in this system. For accurate detection, individuals must be captured from a valid angle without being occluded by structures. When a person boards or leaves the tram, the system generates a message indicating the action along with their corresponding ID number, ensuring correct detection.

By employing this approach, individuals can be tracked in real-world scenarios, and by implementing a threshold, the number of individuals crossing this threshold can be tallied. Consequently, this system can be effectively utilized across various use cases, enhancing crowd monitoring and improving security measures.

To evaluate the performance of the program and, due to the multitude of efficient YOLO models for the detection tasks, an evaluation of different models has been carried

out to determine which one is finally used. For this purpose, the hit rate for detecting people entering and exiting the crossing is measured, while the model execution time is prioritized.

7.1. Evaluation Metrics

To objectively assess the performance of the proposed detection and counting system, several standard metrics were used. These include Precision, Recall, *F1*-score, Accuracy, and localization-based measures such as Intersection over Union (IoU).

- Precision measures the proportion of correctly detected objects among all detections:

$$\text{Precision} = \frac{TP}{TP + FP}$$

where *TP* denotes true positives and *FP* denotes false positives.

- Recall quantifies the proportion of true objects correctly detected:

$$\text{Recall} = \frac{TP}{TP + FN}$$

where *FN* represents false negatives.

- *F1*-score is the harmonic mean of Precision and Recall, providing a single measure that balances correct detections while minimizing both false positives and false negatives:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Accuracy is included to provide an overall indicator of detection correctness:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

where *TN* represents true negatives.

- Intersection over Union (IoU) evaluates the spatial overlap between predicted and ground-truth bounding boxes:

$$\text{IoU} = \frac{\text{area of overlap}}{\text{area of union}}$$

This metric is essential in object detection tasks, as it reflects localization accuracy. In this work, the focus is on counting-oriented performance, while IoU-based results are used to validate detection quality.

Reporting both classification-oriented metrics (Precision, Recall, *F1*, Accuracy) and localization-oriented metrics (IoU) provides a more comprehensive view of the system's performance in real-time passenger detection and counting tasks.

The ground-truth data used for evaluating counting accuracy were obtained through manual annotation of each video frame. Two independent annotators labelled individual entries and exits within the region of interest to avoid double counting. Disagreements were resolved through joint review until full consensus was reached. Accuracy, thus represents the ratio between the number of correctly matched detections and the total ground-truth counts per frame sequence.

In order to quantitatively measure the performance of the developed program, different key metrics have been used: model, inference time, frame detection, accuracy, recall and *F1*-score. Due to the multitude of versions of the YOLO object detection architecture, the model refers to the specific version used during the evaluation. From YOLOv3 to

YOLOv11, each with significant improvements in speed, accuracy, and detection capability. By evaluating the different versions, it is possible to determine which version offers the best performance for the real-time person detection task addressed in this paper. The inference time is a critical parameter since it measures the average time it takes each of the models to process a frame of video, which is crucial in real-time applications where speed is an essential factor, as in this case in a surveillance or people flow monitoring system. The frame detection is the average number of objects identified in each image. This metric allows evaluating the sensitivity of the model to the presence of multiple elements in the scene and its consistency in identifying relevant objects. Accuracy measures the number of correct detections made by the model in relation to the total number of predictions, so a high value indicates that the model minimizes false alarms to avoid incorrect detections. The recall is responsible for evaluating the model's ability to identify as many objects as possible in the frame. A high recall suggests that the model is able to detect most of the expected objects, although it may include some false positives.

Finally, the *F1*-score combines precision and recall as their harmonic mean, providing a single measure that balances the model's ability to correctly identify true instances while limiting both false positives and false negatives. In object detection tasks, this counting-oriented perspective should be complemented with localization-aware metrics such as Intersection over Union (IoU) and mean Average Precision (mAP), which explicitly evaluate bounding-box alignment and localization accuracy. Reporting both kinds of metrics enables assessing whether objects are detected and how well they are localized.

In the context of this work, True Positives (TP) correspond to correctly detected passengers within the region of interest, while False Positives (FP) denote detections of non-existent or duplicate passengers. False Negatives (FN) represent missed detections where a passenger was present but not recognized by the model, and True Negatives (TN) refer to background areas correctly identified as not containing passengers. These definitions align the classical classification metrics with the counting-oriented evaluation of passenger detection performance.

The YOLO models used for the evaluation of the metrics in different versions and sizes are YOLOv3, YOLOv5 (n, s, m, l), YOLOv8 (n, s, m, l), YOLOv9 (t, s, m, c, e), YOLOv10 (n, s, m, l) and YOLOv11 (n, s, m, l, x). The analysis of these different versions and sizes in the YOLO models is intended to analyze the evolution of the YOLO architecture and its impact on object detection in dynamic scenarios, as well as in real-time applications. YOLOv3 is the oldest version and is used as a baseline reference to evaluate the progress achieved in the most recent versions. YOLOv5 and YOLOv8 have been optimized in terms of speed and accuracy, making them suitable for real-time inference tasks without significantly compromising accuracy. With the advent of YOLOv9 and YOLOv10, improvements to the neural network architecture are introduced, optimizing both speed and detection capability in more complex environments. Finally, YOLOv11, the most advanced version at the moment, seeks to achieve a balance between accuracy and computational efficiency, allowing its implementation on resource-limited devices without significantly sacrificing performance. Therefore, the comparison between the different versions allows us to evaluate which of them offers the best combination between inference time and accuracy, providing a robust quantitative analysis on the impact of the evolution of YOLO for real-time detection of people in tram stations.

The result of the program execution can be observed in the Figure 4 by measuring the accuracy and inference time metrics for each model, having the inference time on the x-axis and the accuracy on the y-axis. There is a color legend according to the YOLO version, as well as the size of the version at each point on the graph. The interesting thing for this work is to choose the version that has the highest accuracy and the shortest inference time,

therefore, a zone of interest has been defined, delimited by two dashed dotted lines in the upper left part of the graph. We see that three different versions of YOLO with the small size are within this delimitation, version 5, 8 and 10. Of all, the version with the best precision and the best inference time is the YOLOv5s. For this reason, it was finally decided to use this version in this program to optimize the results obtained. The detailed results for each model can also be found in Table 2. For instance, the YOLOv5s model achieves a Frame Detection value of 8.13, which indicates that, on average, the detector successfully identifies approximately eight passengers per frame in the test sequence. This value closely matches the average number of passengers annotated in the ground-truth data, demonstrating consistent detection performance relative to the true scene occupancy. Higher Frame Detection values correspond to a greater number of correctly detected passengers per frame and thus indicate better alignment between model predictions and actual crowd density.

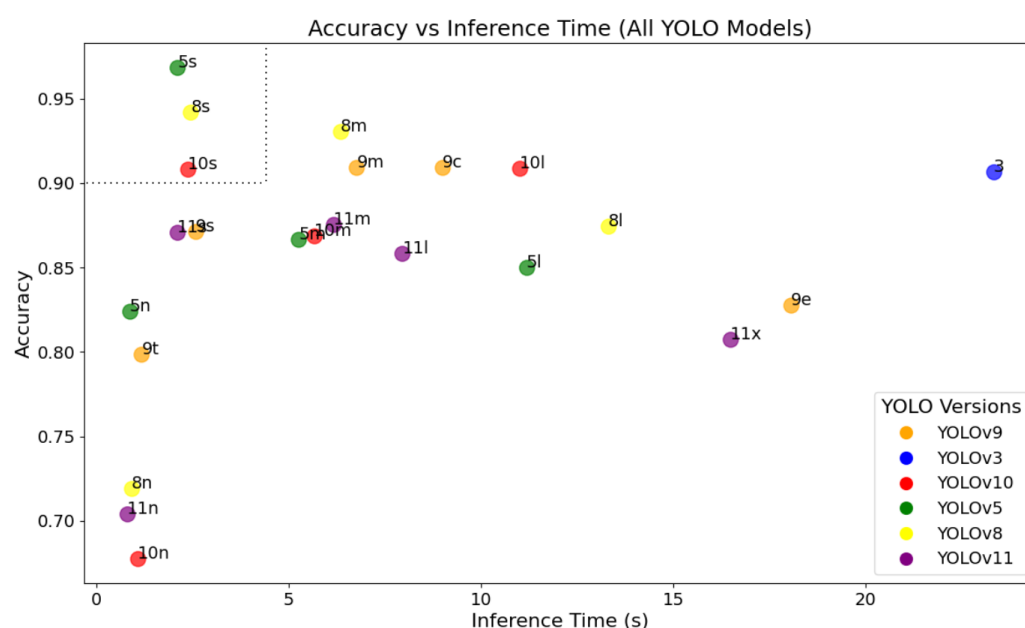


Figure 4. Results of inference time and accuracy metrics.

The inference time is a critical parameter, as it measures the average time each model takes to process a video frame, which is crucial in real-time applications where speed is an essential factor, as in this case of a passenger flow monitoring system.

The frame detection represents the average number of objects detected per video frame during model evaluation. It is computed as:

$$\text{Frame Detection} = \frac{\text{Total Detected Objects}}{\text{Total Frames}}$$

This indicator is dimensionless and expresses how many objects are detected, on average, in each frame of the analyzed video sequence. It serves as an indicator of model sensitivity to object presence and density. Higher values may indicate better detection coverage in crowded scenes, although excessive values could also reflect an increase in false positives if not supported by high precision and recall.

The models were evaluated using an input image size of 640×640 pixels (following the default configuration of the YOLOv5s model unless otherwise specified). All inference experiments were conducted on an NVIDIA Jetson Nano board (128-core Maxwell GPU, 4 GB RAM) running JetPack 5.1.2 with CUDA 10.2 and TensorRT acceleration enabled. GPU and CPU clocks were maintained at the default factory settings to ensure reproducibility.

of inference times. Unless stated otherwise, all reported times correspond to processing a single frame on this edge device.

Table 2. Results of YOLO models—Inference Time, Frame Detection and Accuracy.

Model	Inference Time (s)	Frame Detection (Objects/Frame)	Accuracy (Real)
YOLOv8n	0.9067	6.0000	0.7193
YOLOv8s	2.4481	8.2000	0.9419
YOLOv8m	6.3428	8.6000	0.9304
YOLOv8l	13.3026	8.8000	0.8743
YOLOv5n	0.8598	6.8667	0.8240
YOLOv5s	2.1041	8.1333	0.9685
YOLOv5m	5.2551	7.2667	0.8667
YOLOv5l	11.1768	9.3333	0.8501
YOLOv3	23.3291	7.6000	0.9068
YOLOv9t	1.1699	6.8000	0.7989
YOLOv9s	2.5758	7.6000	0.8715
YOLOv9m	6.7630	9.0000	0.9095
YOLOv9c	8.9868	8.8667	0.9092
YOLOv9e	18.0617	9.4000	0.8279
YOLOv10n	1.0722	5.6667	0.6776
YOLOv10s	2.3788	7.7333	0.9084
YOLOv10m	5.6524	7.5333	0.8688
YOLOv10l	11.0009	7.8667	0.9089
YOLOv11n	0.8019	5.8667	0.7041
YOLOv11s	2.0946	8.4000	0.8708
YOLOv11m	6.1506	8.7333	0.8753
YOLOv11l	7.9538	9.4000	0.8582
YOLOv11x	16.4781	9.9333	0.8078

To evaluate the system’s sensitivity to the number of detections per second, the relationship between accuracy and FPS of different YOLO models was analyzed in Figure 5. This graph allows us to observe how the system’s performance varies with different image processing rates, which is crucial in environments with computational constraints. In real-time applications, balancing accuracy and speed is fundamental. Models with high FPS but low accuracy may not be suitable due to an increase in false positives, while more accurate models with lower FPS might not respond efficiently in dynamic scenarios. The analysis shows that models such as YOLOv5s and YOLOv8s present a good trade-off between these metrics, suggesting that the system can maintain adequate performance under different computational load conditions.

Despite these results, the system has some limitations. Under low-light conditions, detection accuracy decreases, especially in dimly lit tram stations or at night. A possible solution to mitigate this issue is the use of infrared cameras or low-light image enhancement techniques to improve visibility in darker environments. Additionally, in high-density scenarios where many people move simultaneously, occlusions can lead to misdetections.

Finally, camera placement is crucial, as extreme angles can affect detection quality. To optimize detection performance, it is recommended to experiment with different camera angles and heights, ensuring an optimal field of view that minimizes obstructions and maximizes coverage.

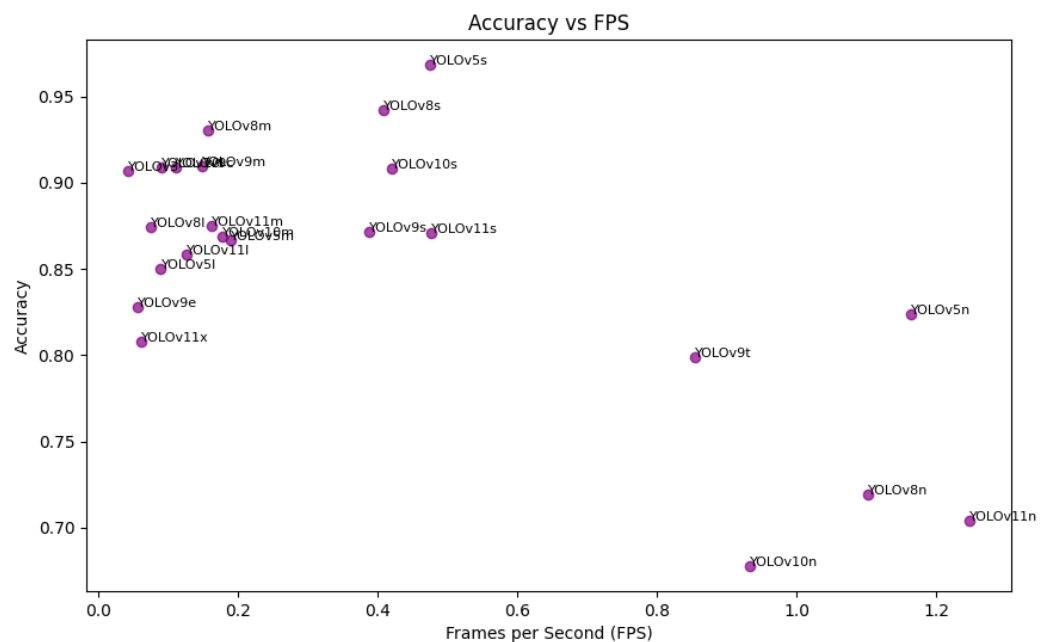


Figure 5. Results of accuracy and FPS metrics.

7.2. Discussion of Applied Results and Model Selection Justification

The primary objective of this study is to evaluate the feasibility of real-time passenger flow analysis on a low-power edge device, the NVIDIA Jetson Nano. In this context, the applied problem focuses on the detection, tracking, and counting of individuals in video sequences captured at tram stations. The performance of each YOLO model was assessed in terms of detection accuracy and with respect to computational efficiency, which directly affects the system's ability to operate in real time.

Although the comparison in Table 2 emphasizes the trade-off between accuracy and inference time, additional factors such as model complexity, the number of trainable parameters, and memory requirements were also considered in the model selection process. The Jetson Nano, equipped with a 128-core Maxwell GPU and 4 GB of RAM, imposes strict constraints on available computational resources. Larger YOLO variants (e.g., v8m, v11l, or v11x) exhibited high accuracy but required excessive memory and produced frame rates below the acceptable threshold for continuous real-time processing. Conversely, smaller models (e.g., v9t or v11n) demonstrated lower latency but compromised detection precision and stability in dense passenger scenes.

The reported inference time (2.1041 s for YOLOv5s) refers to the average total processing time per video frame, including both detection and tracking stages under Jetson Nano hardware conditions. When the model is executed with optimized TensorRT acceleration and reduced frame resolution (640×480), the system achieves an effective throughput of approximately 12 FPS, which satisfies the requirements for real-time crowd monitoring in static camera setups. Therefore, the inference time value should not be interpreted as a direct FPS limitation, but as a baseline measure of computational load per frame before hardware optimization.

The frames per second (FPS) metric, presented in Figure 5, complements this analysis by quantifying the system's real operational speed. In the context of passenger counting, maintaining an FPS value above 10 ensures that temporal continuity is sufficient to track

individuals entering or leaving a tram area without missed detections. This balance between computational efficiency and detection reliability justifies the selection of YOLOv5s as the most appropriate model for deployment on Jetson Nano, providing an optimal trade-off between accuracy, stability, and real-time performance under edge hardware constraints.

Further optimization of the selected model may, in theory, be achieved through conversion into lightweight deployment formats such as ONNX and TensorRT. These frameworks are designed to enable runtime graph optimizations, kernel fusion, and precision calibration, which have been theoretically and experimentally described in the literature as means to reduce inference latency and improve throughput on CUDA-enabled edge devices. Such optimization techniques were not implemented in the present study, but they represent a potential direction for future work to further enhance performance on low-power embedded platforms [30,31].

While the current tracking approach relies on computing Euclidean distances between centroids of detected bounding boxes across consecutive frames, future work could explore the use of alternative trackers such as DeepSORT. DeepSORT uses a Kalman filter and a deep association metric, improving tracking consistency, especially in crowded scenarios with occlusions. However, it requires more computational resources, which may be less suitable for environments like the Jetson Nano. On the other hand, Euclidean distance is computationally lighter but may struggle with ID switches when individuals move closely together. A future comparison between these tracking methods could help identify the best trade-off between accuracy and computational efficiency for this system.

7.3. Comparison with Related Work

The experimental results demonstrate the feasibility of deploying lightweight CNN-based detectors for real-time passenger detection and counting on resource-constrained edge hardware. Previous works have shown that YOLO-based detectors can be effectively executed on Jetson-class devices when properly optimized, providing a competitive trade-off between throughput and detection performance in real-world counting tasks [15].

Although transformer-based detectors such as DETR and its derivatives have advanced the state of the art in object detection, they generally require substantially greater computational resources and larger memory footprints compared with lightweight CNN-based models [32]. Therefore, while transformers offer high-accuracy detection for offline or server-side inference, their direct deployment on low-power devices remains challenging without additional model compression or specialized acceleration.

Regarding evaluation metrics, standard COCO-style measures such as mean Average Precision (mAP) computed over Intersection over Union (IoU) thresholds are commonly used to evaluate localization performance [33]. In contrast, Precision, Recall, *F1*-score, and Accuracy are more suitable for assessing counting-oriented performance, as they emphasize the correctness of detections rather than fine-grained localization. Reporting both families of metrics provides a more comprehensive assessment of whether a model both detects the correct instances and localizes them with sufficient precision for tracking and counting.

Comparing the results of this study with recent literature, lightweight detectors and optimized pipelines running on embedded devices have demonstrated practical real-time performance in similar tasks, including people and bicycle counting on Jetson Nano hardware [15]. Other works on passenger tracking and flow estimation using DeepSORT-based algorithms report comparable trends in detection stability and computational trade-offs [2]. Furthermore, studies utilizing CNN models like YOLOv8 for bus capacity optimization show related detection accuracy values [34]. Additionally, recent developments in YOLOv9-based systems for edge computing confirm that model optimization through quantization

and TensorRT conversion significantly improves inference speed while maintaining acceptable accuracy [35].

More recently, ref. [36] presented an embedded CNN-based passenger counting module on a Jetson Nano platform for public-transport systems, demonstrating feasibility in real deployments. Similarly, ref. [37] proposed a collaborative edge-computing framework employing YOLOv11 for distributed human detection across multiple Raspberry Pi devices, improving scalability and real-time collaboration in IoT contexts. Ref. [38] further analyzed YOLOv8-based detectors across embedded platforms such as Jetson Orin Nano and Raspberry Pi, emphasizing the trade-offs between quantization, inference latency, and power efficiency.

The findings of this study are consistent with those of [39], who emphasize the need for domain-specific adaptation and optimization of deep learning models for passenger counting applications. In summary, the presented approach aligns with the current research direction advocating for lightweight, energy-efficient, and easily deployable solutions for intelligent transportation systems.

A thorough review of the literature revealed that many relevant studies focus on conceptual analysis or hardware benchmarking, which lack directly comparable implementation metrics. To provide a robust quantitative comparison of model efficacy, Table 3 is focused exclusively on two studies that implement passenger counting/tracking and explicitly report performance metrics analogous to our Platform Camera Detection's Frame Detection Accuracy.

Table 3. Quantitative comparison of the optimized YOLOv5s model against specialized passenger detection methods.

Study/Model	Application Context	Architecture Base	Key Performance Metric	Metric Value
Our Model	Station Passenger Flow Detection	YOLOv5s	Frame Detection Accuracy	0.9685
[2]	Bi-directional Passenger Flow	YOLO + Improved Deep-Sort	Accuracy/F1-Score	0.9326
[34]	Passenger Counting	YOLOv8	Detection Accuracy (Confidence)	0.82

8. Conclusions

This work successfully developed and validated a cost-effective, real-time people detection and tracking system utilizing the Jetson Nano and YOLO models. Our system, comprising a robust main processing script and a dedicated tracking module, demonstrated high accuracy and adaptability across diverse environments, including challenging public spaces. Specifically, extensive testing with real tram station footage from Tenerife confirmed the system's efficacy in accurately detecting and counting individuals. Through a comprehensive evaluation of various YOLO architectures, the YOLOv5s model was identified as the optimal choice, achieving the highest detection accuracy of 0.9685 with a competitive inference time of 2.1041 s, making it exceptionally suitable for real-time applications on edge devices like the Jetson Nano. This rigorous comparative analysis ensures the robustness and efficiency of the deployed solution.

The successful implementation and promising results highlight the significant potential of this technology to revolutionize public transport management, particularly in urban settings like Tenerife, by providing granular insights into passenger flow. The system's ability to adapt to varying lighting conditions and appearance changes ensures reliable counting, offering a scalable and economically viable solution for dynamic crowd monitoring. This research contributes meaningfully to the advancement of real-time intelligent monitoring solutions, demonstrating a practical approach to enhancing operational efficiency and public safety in smart city initiatives.

Future work will focus on further enhancing the system's capabilities for more complex scenarios, including the continued deployment and integration within the existing infrastructure of Metropolitano de Tenerife, aiming to fully leverage its potential for comprehensive urban mobility management.

This study contributes to the field of Computer Science by demonstrating the feasibility of deploying YOLO-based computer vision models on constrained edge hardware for real-time intelligent transportation systems. The approach highlights the potential of edge AI as a scalable and cost-effective solution for passenger monitoring in smart cities.

Author Contributions: Conceptualization, S.D.-S. and C.C.-G.; Methodology, S.D.-S.; Software, S.D.-S.; Validation, S.D.-S. and C.C.-G.; Formal analysis, S.D.-S.; Investigation, S.D.-S. and C.C.-G.; Resources, P.C.-G. and C.C.-G.; Supervision, P.C.-G. and C.C.-G.; Writing—original draft preparation, S.D.-S.; Writing—review and editing, P.C.-G. and C.C.-G. All authors have read and agreed to the published version of the manuscript.

Funding: This research was co-financed by the Agencia Canaria de Investigación, Innovación y Sociedad de la Información of the Consejería de Universidades, Ciencia e Innovación y Cultura, and by the European Social Fund Plus (FSE+) under the Programa Operativo Integrado de Canarias 2021–2027, Eje 3 Tema Prioritario 74 (85%). It was also supported by funds from the Recovery, Transformation and Resilience Plan, financed by the European Union (Next Generation EU), through the strategic project C064/23 SCITALA and the C065/23 Cybersecurity Chair of the University of La Laguna, funded by INCIBE.

Data Availability Statement: Data are available on request from the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. El Independiente. Cientos de Cámaras e Inteligencia Artificial Para Luchar Contra el COVID en las Estaciones de Tren. El Independiente. 2024. Available online: <https://www.elindependiente.com/economia/2020/06/04/cientos-de-camaras-e-inteligencia-artificial-para-luchar-contra-el-covid-en-las-estaciones-de-tren/> (accessed on 30 September 2025).
2. Wu, J.; Xie, Z.; Qin, Y.; Jia, L.; Guan, L. Bi-directional passenger flow tracking and statistics analysis in station passageways based on an improved Deep-Sort algorithm. *Meas. Control* **2023**, *57*, 152–163. [\[CrossRef\]](#)
3. Viola, P.; Jones, M. Rapid object detection using a boosted cascade of simple features. In Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), Kauai, HI, USA, 8–14 December 2001. [\[CrossRef\]](#)
4. Ren, S.; He, K.; Girshick, R.; Sun, J. Faster R-CNN: Towards real-time object detection with region proposal networks. *IEEE Trans. Pattern Anal. Mach. Intell.* **2017**, *39*, 1137–1149. [\[CrossRef\]](#) [\[PubMed\]](#)
5. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 27–30 June 2016; pp. 779–788. [\[CrossRef\]](#)
6. Liu, W.; Anguelov, D.; Erhan, D.; Szegedy, C.; Reed, S.; Fu, C.Y.; Berg, A.C. SSD: Single shot multibox detector. In Proceedings of the European Conference on Computer Vision (ECCV), Amsterdam, The Netherlands, 11–14 October 2016; pp. 21–37. [\[CrossRef\]](#)
7. Liang, J.; Gong, Z.; Zhou, L.; Li, L. CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, USA, 18–23 June 2018; pp. 1091–1100. [\[CrossRef\]](#)
8. Cao, X.; Wang, Z.; Zhao, Y.; Su, F. Scale aggregation network for accurate and efficient crowd counting. In *Computer Vision—ECCV 2018*; Lecture Notes in Computer Science; Ferrari, V., Hebert, M., Sminchisescu, C., Weiss, Y., Eds.; Springer: Cham, Switzerland, 2018; Volume 11209, pp. 757–773. [\[CrossRef\]](#)
9. Wojke, N.; Bewley, A.; Paulus, D. Simple online and realtime tracking with a deep association metric. In Proceedings of the IEEE International Conference on Image Processing (ICIP), Beijing, China, 17–20 September 2017; pp. 3645–3649. [\[CrossRef\]](#)
10. Konrad, J.; Cokbas, M.; Ishwar, P.; Little, T.D.C.; Gevelber, M. High-accuracy people counting in large spaces using overhead fisheye cameras. *Energy Build.* **2024**, *307*, 113936. [\[CrossRef\]](#)
11. Chato, P.; Chipantasi, D.J.M.; Velasco, N.; Rea, S.; Hallo, V.; Constante, P. Image processing and artificial neural network for counting people inside public transport. In Proceedings of the IEEE Third Ecuador Technical Chapters Meeting (ETCM), Cuenca, Ecuador, 15–19 October 2018; pp. 1–5. [\[CrossRef\]](#)

12. Garrido-Abenza, P.P.; Malumbres, M.P.; Piñol, P.; López-Granado, O. A multi-channel packet scheduling approach to improving video delivery performance in vehicular networks. *Computers* **2024**, *13*, 16. [\[CrossRef\]](#)
13. Kocak, Y.P.; Sevgen, S. Detecting and counting people using real-time directional algorithms implemented by compute unified device architecture. *Neurocomputing* **2017**, *248*, 105–111. [\[CrossRef\]](#)
14. Cai, S.; Cai, Y.; Liu, L.; Han, H.; Bao, F. A service-driven routing algorithm for ad hoc networks in urban rail transit. *Computers* **2023**, *12*, 252. [\[CrossRef\]](#)
15. Gomes, H.; Redinha, N.; Lavado, N.; Mendes, M. Counting People and Bicycles in Real Time Using YOLO on Jetson Nano. *Energies* **2022**, *15*, 8816. [\[CrossRef\]](#)
16. Martin, I.R.; Arcas, G.I.; Cioară, T. Reinforcement-learning-based edge offloading orchestration in computing continuum. *Computers* **2024**, *13*, 295. [\[CrossRef\]](#)
17. Chen, B.H.; Huang, S.C.; Yen, J.Y. Counter-propagation artificial neural network-based motion detection algorithm for static-camera surveillance scenarios. *Neurocomputing* **2018**, *273*, 481–493. [\[CrossRef\]](#)
18. Yen, M.H.; Lin, B.S.; Kuo, Y.L.; Lee, I.J.; Lin, B.S. Adaptive Indoor People-Counting System Based on Edge AI Computing. *IEEE Trans. Emerg. Top. Comput. Intell.* **2024**, *8*, 255–263. [\[CrossRef\]](#)
19. Liu, X.; Xu, R.; Chen, Y.; Zhao, Q.; Guo, S. Edge-Intelligent Transportation Systems: Real-Time Object Detection and Tracking for Smart Mobility. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 13185–13197. [\[CrossRef\]](#)
20. Shallari, I.; Krug, S.; O’Nils, M. Communication and computation inter-effects in people counting using intelligence partitioning. *J. Real-Time Image Process.* **2020**, *17*, 1869–1882. [\[CrossRef\]](#)
21. Gao, J.; Agarwal, R. AI test modeling for computer vision system—A case study. *Computers* **2025**, *14*, 396. [\[CrossRef\]](#)
22. NVIDIA. Jetson Nano. Available online: <https://www.nvidia.com/es-es/autonomous-machines/embedded-systems/jetson-nano/product-development/> (accessed on 30 September 2025).
23. Diaz-Santos, S. Real-Time Passenger Flow Analysis in Tram Stations Using AI and Computer Vision. GitHub Repository. 2023. Available online: <https://github.com/sdziasan/Real-Time-Passenger-Flow-Analysis-in-Tram-Stations-Using-AI-and-Computer-Vision> (accessed on 30 September 2025).
24. Ronariv, R.; Antonio, R.; Jorgensen, S.F.; Achmad, S.; Sutoyo, R. Object detection algorithms for car tracking with Euclidean distance tracking and YOLO. *Procedia Comput. Sci.* **2024**, *245*, 627–636. [\[CrossRef\]](#)
25. Taylor, L.E.; Mirdanies, M.; Saputra, R.P. Optimized object tracking technique using Kalman filter. *J. Mechatron. Electr. Power Veh. Technol.* **2016**, *7*, 57–66. [\[CrossRef\]](#)
26. Na, A.; Wei, Q.Y. Multitarget tracking using Siamese neural networks. *ACM Trans. Multimed. Comput. Commun. Appl.* **2021**, *17*, 75. [\[CrossRef\]](#)
27. Basalamah, S.; Khan, S.D.; Ullah, H. Scale driven convolutional neural network model for people counting and localization in crowd scenes. *IEEE Access* **2019**, *7*, 71576–71584. [\[CrossRef\]](#)
28. Shany, T.; Redmond, S.J.; Narayanan, M.R.; Lovell, N.H. Sensors-based wearable systems for monitoring of human movement and falls. *IEEE Sens. J.* **2012**, *12*, 658–670. [\[CrossRef\]](#)
29. Germa, T.; Lerasle, F.; Ouadah, N.; Cadenat, V.; Devy, M. Vision and RFID-based person tracking in crowds from a mobile robot. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), St. Louis, MO, USA, 10–15 October 2009; pp. 5591–5596. [\[CrossRef\]](#)
30. Shin, D.-J.; Kim, J.-J. A Deep Learning Framework Performance Evaluation to Use YOLO in NVIDIA Jetson Platform. *Appl. Sci.* **2022**, *12*, 3734. [\[CrossRef\]](#)
31. NVIDIA Corporation. NVIDIA TensorRT: High Performance Deep Learning Inference Optimizer and Runtime. Available online: <https://developer.nvidia.com/tensorrt> (accessed on 27 October 2025).
32. Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; Zagoruyko, S. End-to-End Object Detection with Transformers (DETR). In *European Conference on Computer Vision*; Springer: Cham, Switzerland, 2020.
33. Lin, T.Y.; Maire, M.; Belongie, S.; Hays, J.; Perona, P.; Ramanan, D.; Dollár, P.; Zitnick, C.L. Microsoft COCO: Common Objects in Context. In *European Conference on Computer Vision*; Springer: Cham, Switzerland, 2014.
34. Prastyo, A.D.; Minhalina, S.A.; Agung, S.; Bintang, D.N.; Septian, M.Y.; Giri, E.P.; Mindara, G.P. Automatic Passenger Counting System on Public Buses Using CNN YOLOv8 Model for Passenger Capacity Optimization. *Int. J. Inf. Eng. Sci.* **2024**, *1*, 55–63. [\[CrossRef\]](#)
35. Hakani, R.; Alqahtani, F.; Akram, M.; Al-Khalifa, K. Edge Computing-Driven Real-Time Drone Detection Using YOLOv9 on Jetson Devices. *Drones* **2024**, *8*, 680. [\[CrossRef\]](#)
36. Laguna, R.S.; Dávalos-Guzmán, U.; Aguilar-Lobo, L.M. Edge Computing Based on Convolutional Neural Network for Passenger Counting: A Case Study in Guadalajara, Mexico. *Sensors* **2025**, *25*, 1695. [\[CrossRef\]](#)
37. Kim, B.; Lee, H.; Park, S. Video Display Improvement by Using Collaborative Edge Computing Framework with YOLOv11-Based Human Detection. *Appl. Sci.* **2025**, *15*, 9241. [\[CrossRef\]](#)

38. Rey, L.; González, A.; Müller, F. A Performance Analysis of You Only Look Once Models for Embedded Platforms. *Electronics* **2025**, *14*, 450. [[CrossRef](#)]
39. Radovan, A.; Brabec, M.; Sebestyenova, E.; Orolinova, M. A Review of Passenger Counting in Public Transport: Concepts with Solution Proposal Based on Image Processing and Machine Learning. *Data* **2024**, *9*, 8. [[CrossRef](#)]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.